

→ The control memory address register specifies the address of the microint (MI)

→ control memory is assumed to be ROM
can control information permanently stored

→ control register holds the microint fetches control word from the memory.

→ while executing MI, the next address is computed in the next address generator.

UNIT - V

1) Different types of interrupts: hardware, software, maskable and non-maskable, vectored and non-vectored.

Interrupts are signals that temporarily stop the CPU's current execution so that it can respond to an urgent event or request.

Hardware Interrupts:- If the signal for the processor is from the hardware device or external device.

Ex, From keyboard Alt+ctrl+delete this pressing of key in keyboard will generate a signal, such as interrupts are called hardware interrupts.

Types of Hardware Interrupts:-
→ Maskable Interrupts:- The hardware interrupt can be delayed when a much higher priority interrupt has occurred to the processor.

→ Nonmaskable Interrupts:- The hardware interrupt which cannot be delayed and should be processed by the processor immediately.

Software Interrupts:-

The interrupt that is caused by any internal interrupt system of the computer system is known as a software interrupt. It is also of two types.

Normal Interrupt:-

- The interrupts that are caused by

software instructions. $a=1$ $a/b=1/b$
 $b=0$

Exception:- unplanned interrupts which are produced during the execution of some program.

Vectored Interrupts:- It is an interrupt in which the address of the interrupt service routine (ISR) is known in advance.

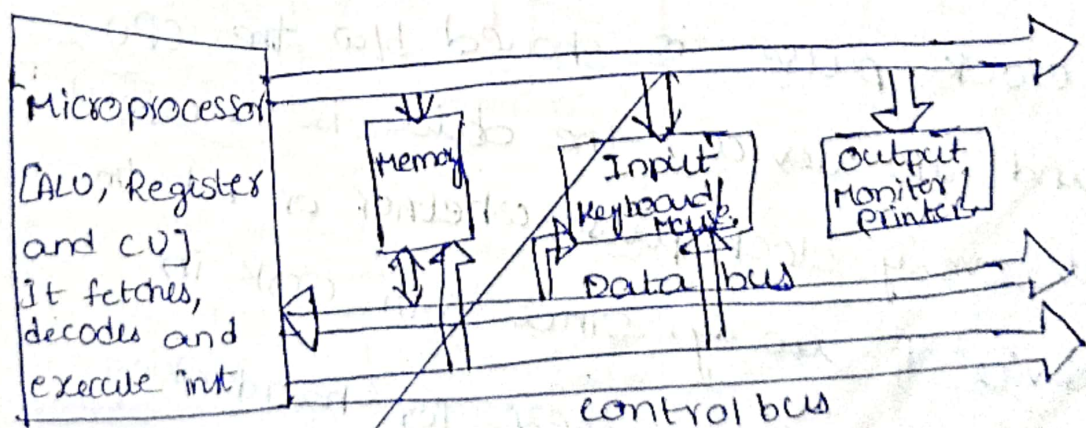
Ex:- TRAP RST 7.5, RST 6.5 etc.
In other words, when the interrupt occurs, the CPU automatically jumps to a predefined memory location (vector address) where ISR starts.

Non- vectored interrupt:- is an interrupt in which the address of the ISR is not fixed. The CPU must be informed by the interrupting device or software where the ISR is located.

→ when the interrupt occurs, the CPU acknowledges the interrupt. The ISR address is supplied by the device or determined by software.

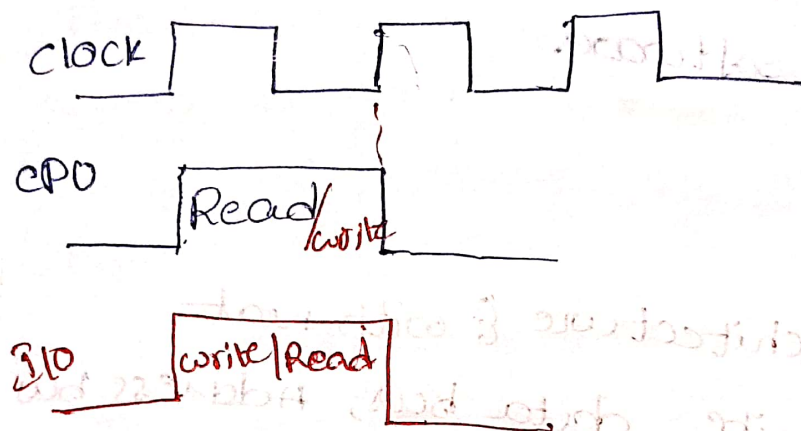
Ex:- INTR

- System bus architecture & with neat diagrams. Describe data bus, Address bus and control bus.



Describe the process of synchronous and asynchronous data transfer in I/O operation.

In synchronous data transfer, the I/O device and CPU are synchronized by a common clock signal. The data transfer occurs at fixed, predefined intervals determined by this clock.



A clock pulse is shared b/w the CPU and I/O device; the data is transferred at every clock pulse, whether or not the device is ready. Since both work in sync, there is no need for handshaking signals like "ready" or "acknowledge".

④ This is not suitable for slow I/O devices

Asynchronous data transfer:-

In asynchronous data transfer, there is "no common clock" b/w the CPU and the I/O device. The data transfer depends on "control signals" indicating each side is ready.

The two approaches used in asynchronous data transfer are

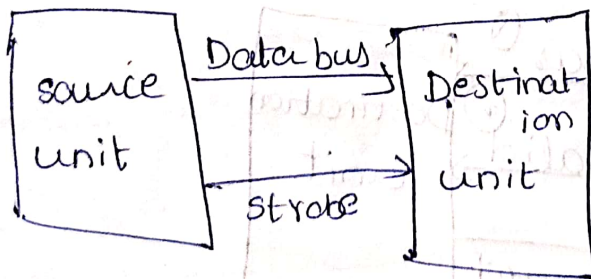
1. Strobe

2. Handshaking

→ Source initiated

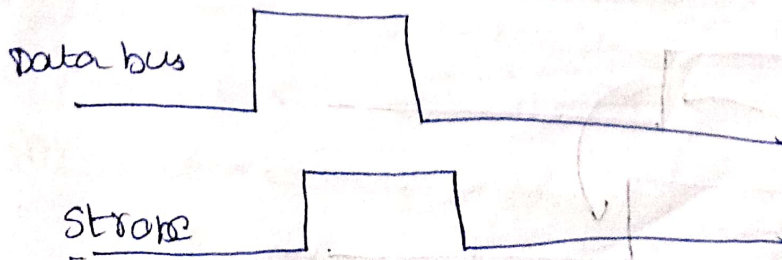
strobe for data transfer

ii) Source initiated strobe for data transfer:-

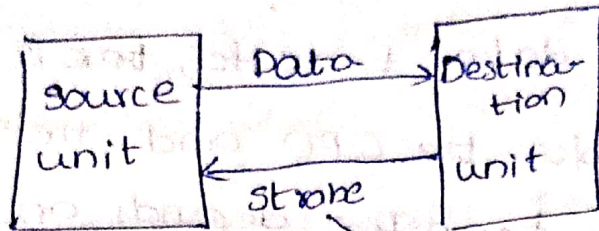


Block diagram.

Timing diagram:-

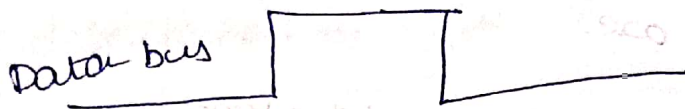
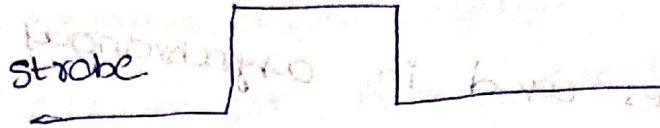


1(i) Destination initiated strobe for data transfer



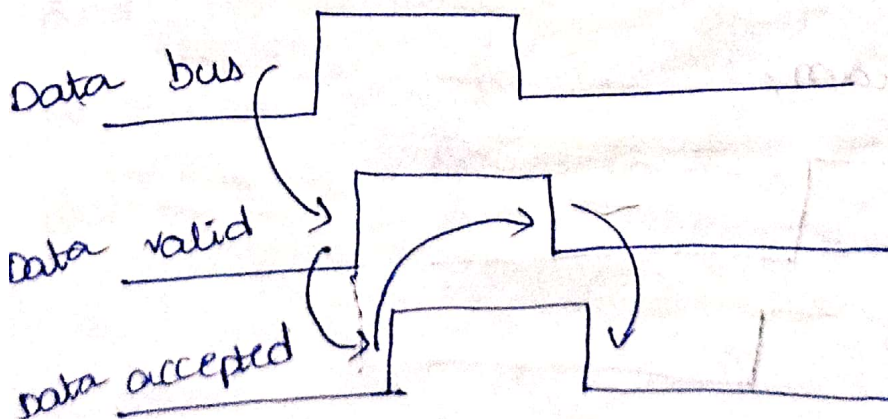
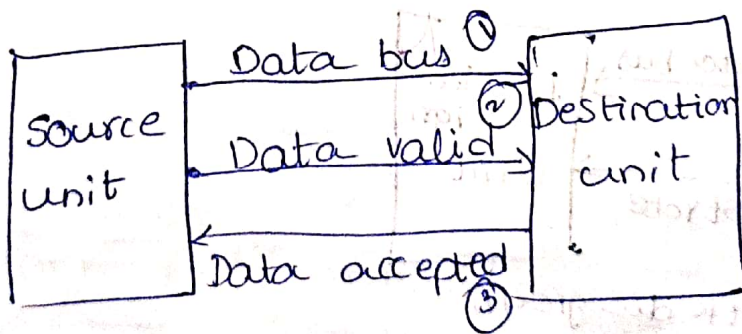
ready for accepting data

Timing diagram



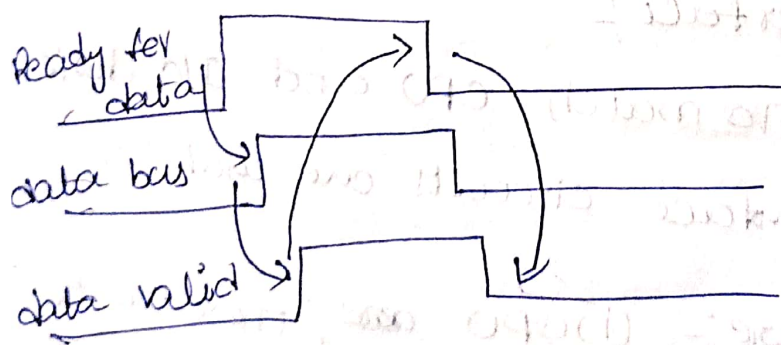
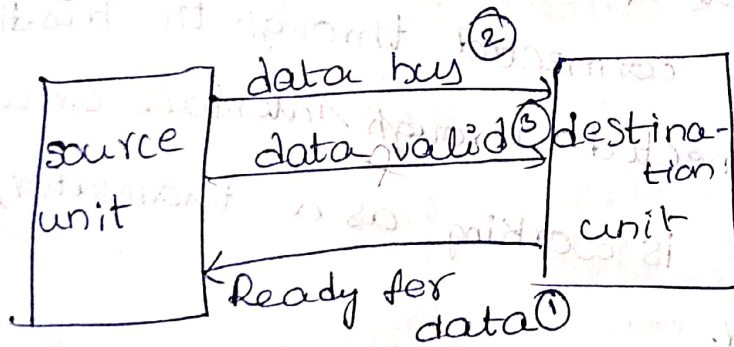
2) Handshaking:- To provide acknowledgement to source and destination.

2(i):- source initiated handshaking for data transfer



Source sends data through data bus. After some delay it sends data valid signal representing valid data is presented in the data bus. After some delay and successful completion of accepting data, destination sends "data accepted signal" to the source. After this the data valid is disabled and data bus doesn't contain any valid data. The data accepted signal is also disabled.

2(ii) Destination Initiated handshaking for data transfer:-



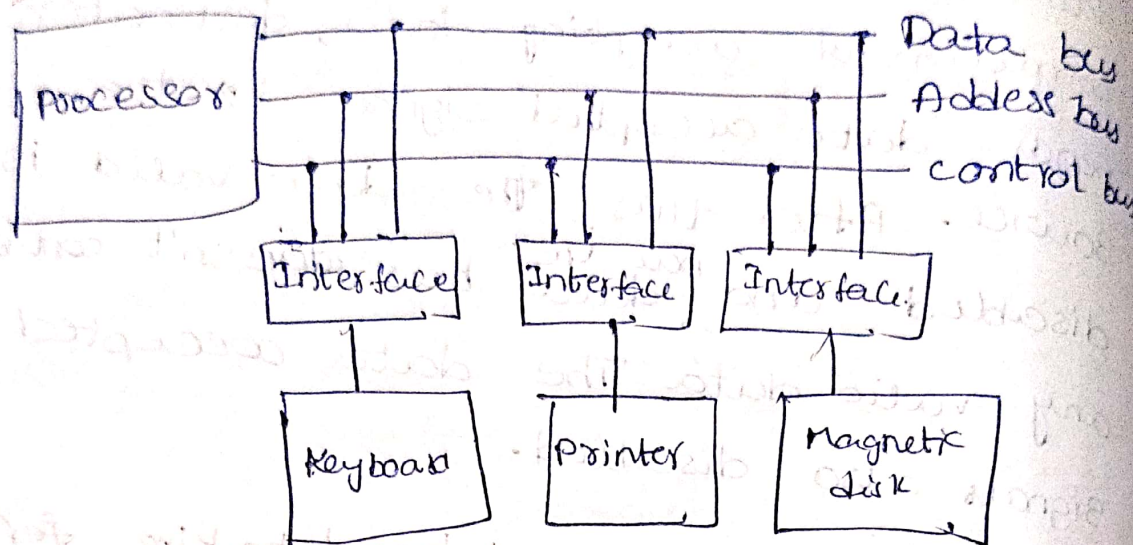
→ Destination sends ready for accept data.

(i) Source transfers data through data bus and enables data valid signal after some delay.

(ii) After successful reception of data, destination disables ready for data and source disables data valid signal.

→ with a block diagram explain the need of a parallel I/O interface.

Q) Explain need for I/O interface



when we connect peripheral devices with CPU they are not connected through the bus directly. They are connected through interface circuit. The interface is working as a translator, as a converter.

Need for interface:-

1) Speed :- To match CPU and I/O device speed interface circuit are used.

2) Device nature:- (1) CPU are electrical devices, and keyboard is electro-mechanical device, magnetic disk is electro-magnetic. The signals from these devices are different. So for the conversion of signals, interface circuit are used.

(3) Data format:-

CPU - 32 bit or 64 bit

Keyboard - 8 ~~bits~~ byte

Printer, Magnetic disk - formats are different.

4) Peripheral logic matching

Keyboard, printer, monitor etc. devices logic is different. CPU can't handle with single logic or can't modify every time. At this time interface cks. are used.

Q:- with a block diagram explain the organisation of Parallel I/O Interface.

Interface circuits are two types.

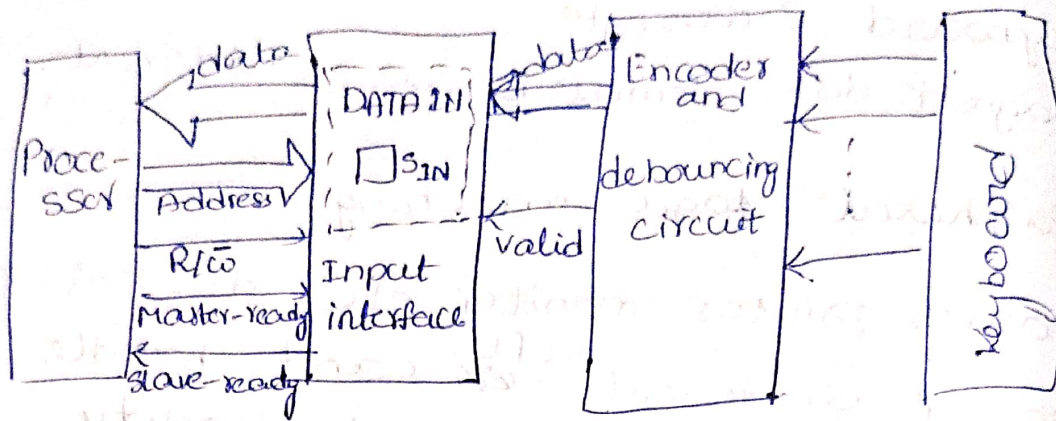
(1) Parallel Port (2) Serial port.
↳ 1 bit at a time

In parallel port transfer of data can be many bits at a time simultaneously. So the fast transmission of data is possible. Ex:- Printer

Interfacing devices one side is connected to data, address and control bus and other side is connected to data path and input/output devices.

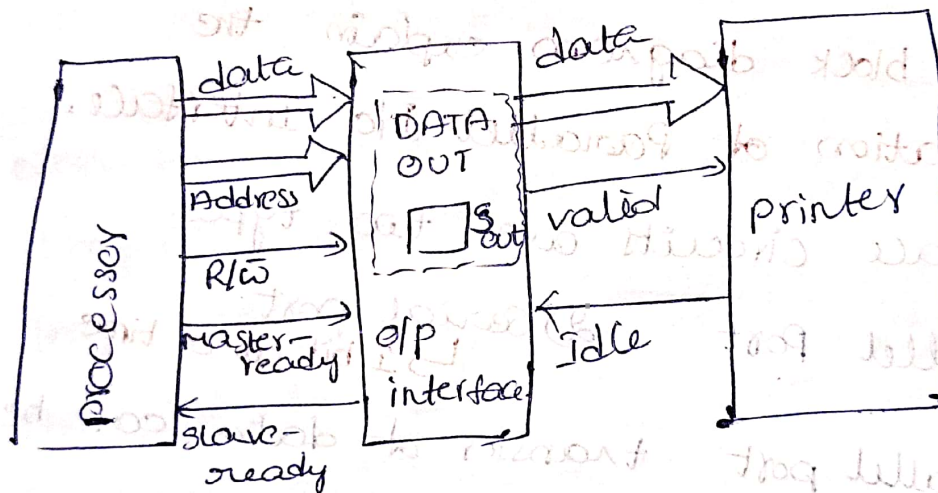
↓
transfers
data b/w interface
and I/O

Keyboard to processor connection (input port)

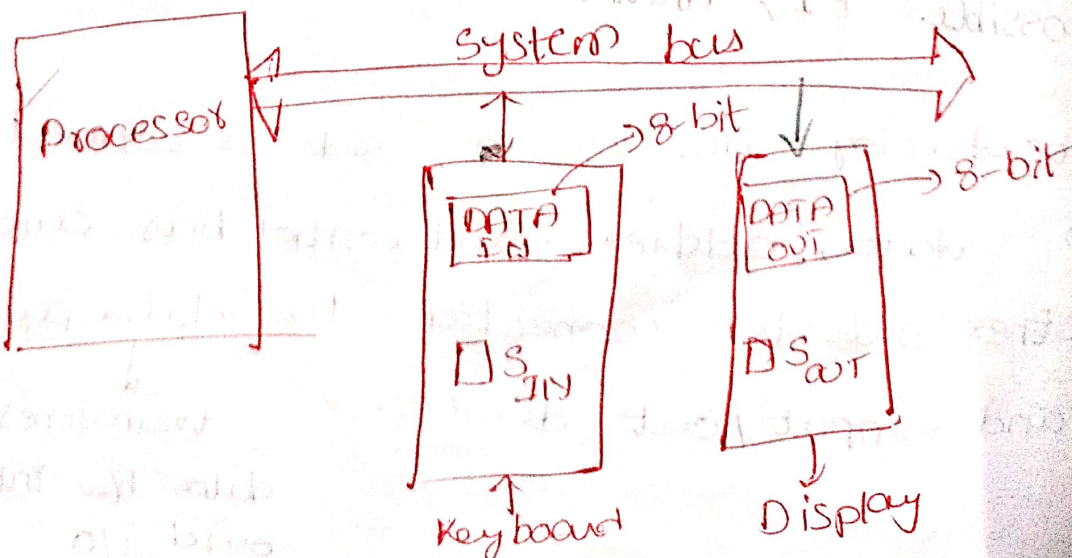


S_{IN} - status flag -

Printer to processor connection (output port)



OR



Q:- Explain the role of status register, control register and buffer register in an I/O interface.

In an I/O interface, the status register, control register and buffer register play crucial roles in coordinating data transfer b/w the CPU and peripheral devices.

① status register:- Provides information about the current state of the I/O device, such as whether it is ready for data transfer, busy or encountered an error. The CPU checks this register before initiating any operation to ensure the device is prepared for communication.

② control register:- is used by the CPU to send control signals or commands to the I/O device, such as selecting b/w read or write operations, enabling or disabling

interrupts, and starting or stopping data transfer.

③ Buffer register: serves as temporary storage for data being transferred b/w CPU and I/O device. During i/p operations, it holds data received from the device until CPU reads it and during o/p operations, it stores data from CPU before sending it to the device.

* Define Bus arbitration?

Bus arbitration is the process of determining which device gets control of the system bus when multiple devices request it simultaneously. Once the bus is granted to the highest priority device, it completes its data transfer and then, control is released for the next request.

* List any two standard I/O interfaces commonly used in computers.

① USB (Universal System Bus)

② SCSI (Small Computer System Interface)

→ what is the role of handshaking signal in data transfer?

A:- Its role is to synchronize the exchange of data b/w two devices. It ensures that both the sender and receiver are ready before data is transmitted, preventing data loss or errors by sending ready or acknowledge signal, confirming that data can be safely sent or received.

→ Give two examples of processors that support vectored interrupts?

① Intel 8085 microprocessor:- supports vectored interrupts such as RST 7.5, RST 6.5, and RST 5.5, each having fixed memory location for the Interrupt service routine (ISR).

(ii) Intel 8086 microprocessor:- where the interrupt type number is used to calculate the address of the ISR stored in the Interrupt vector Table (IVT).

→ State two advantages of using interrupt-driven I/O?

① Efficient CPU utilization - The CPU can perform other tasks instead of constantly checking the device status, improving overall system efficiency.

② Faster response to I/O device: The CPU is immediately alerted when an I/O device needs attention, reducing waiting time and improving performance.

→ Differentiate b/w synchronous and asynchronous buses.

Synchronous bus

1. Uses common clock signal shared by all devices.

2. Data transfer occurs at regular, fixed intervals.

3. Simpler to design but limited by clock speed.

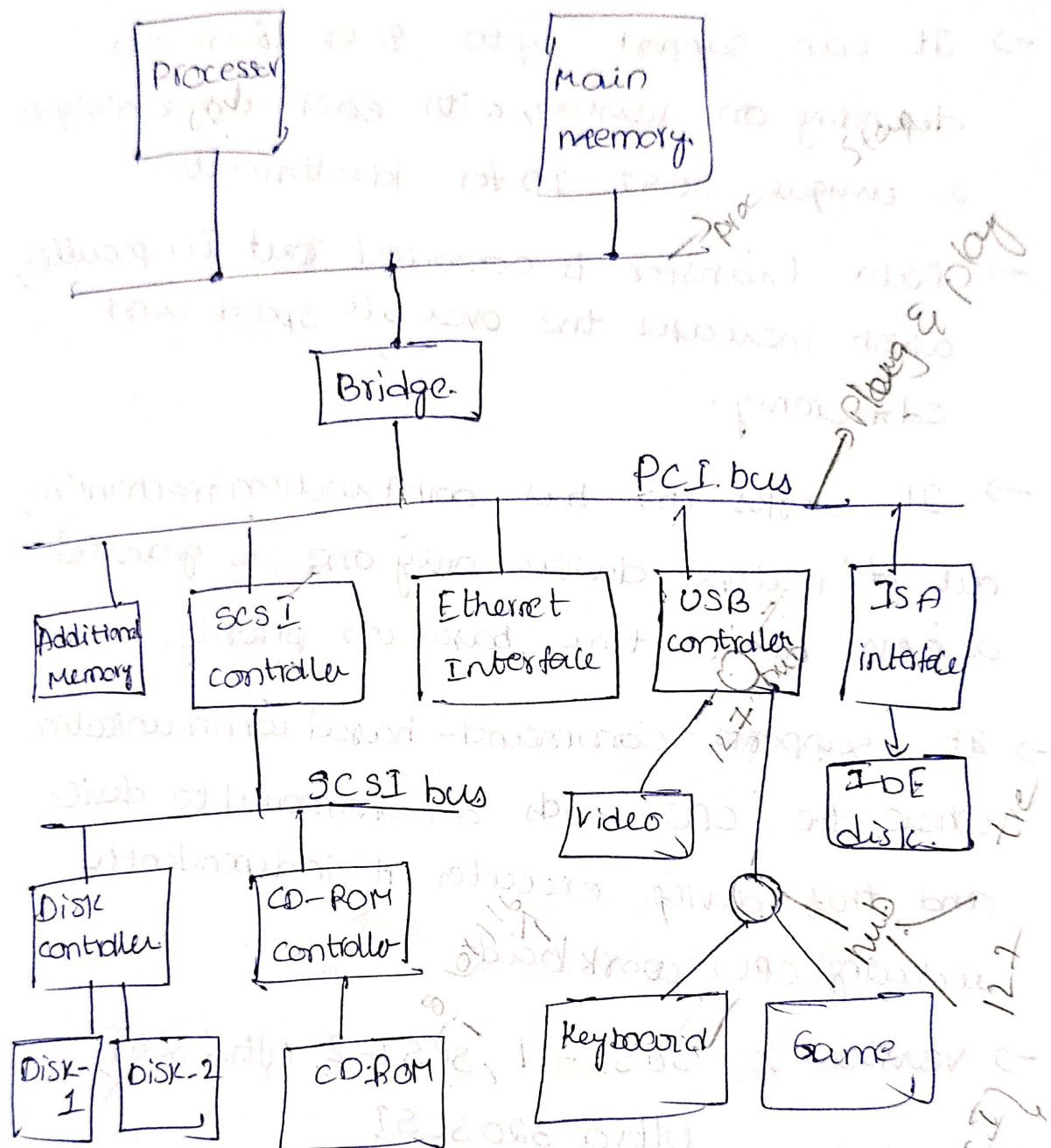
Asynchronous bus

1. Does not use a common clock, timing is controlled by handshaking.

2. Data transfer occurs whenever both sender and receiver are ready.

3. More complex due to control signals.

→ Explain the SCSI bus standard.



PCI- peripheral component interconnect

SCSI:- small computer system interface bus

standard is a widely used parallel communication standard that defines how computers and peripheral devices exchange data.

→ It provides a high-speed, flexible method

for connecting multiple devices such as hard disk, CD/DVD drives, scanners and printers.

→ It can support upto 8 or 16 devices depending on version, with each device assigned a unique SCSI ID for identification.

→ Data transfer is carried out in parallel, which increases the overall speed and efficiency.

→ It works on bus arbitration mechanism, out of multiple devices only one is granted access at a time based on priority.

→ It supports command-based communication where the CPU sends a command to device and that device executes it independently reducing CPU workload.

→ versions :- SCSI-1, SCSI-2, Ultra SCSI, Ultra 320 SCSI

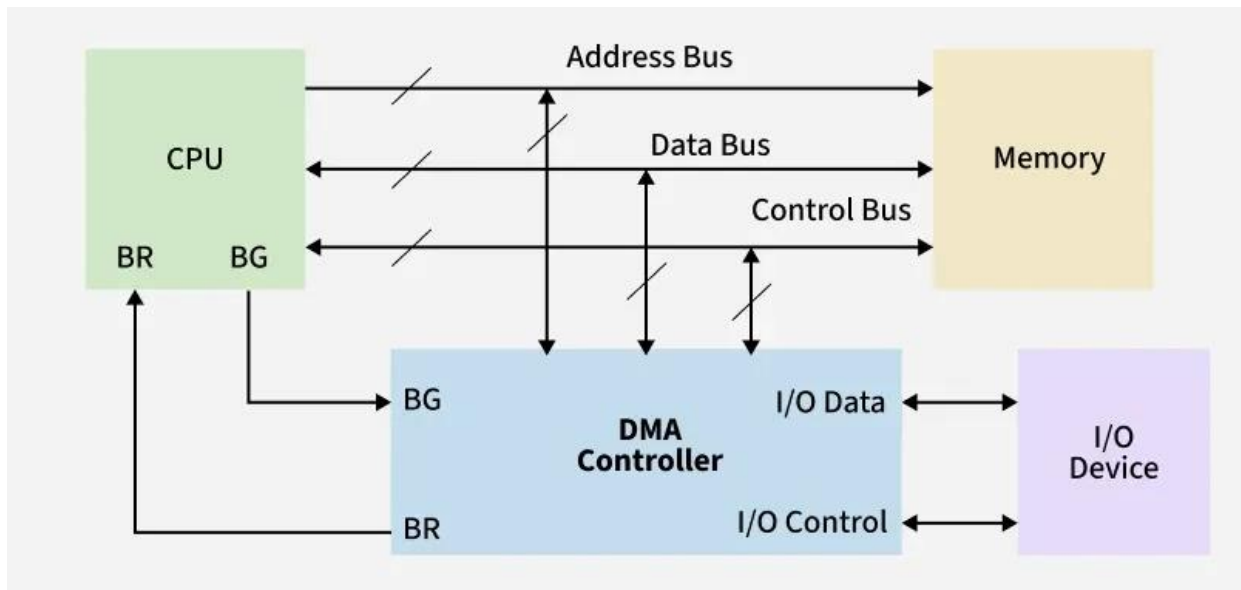
Q: Differentiate between single-bus, multiple-bus, and hierarchical bus structures with examples.

Feature	Single-Bus Structure	Multiple-Bus Structure	Hierarchical Bus Structure
Definition	A single common bus connects the CPU, memory, and all I/O devices.	Two or more buses are used to connect different groups of components to reduce traffic.	A multi-level bus system where buses are organized in layers (e.g., system bus, I/O bus, local bus) based on function and performance.
Data Transfer Path	All components share the same communication path (one bus).	Several buses operate simultaneously for different transfers.	Data is transferred between levels through bridge interfaces.
Speed and Performance	Slower, as only one data transfer can occur at a time (bus contention).	Faster, since multiple transfers can occur in parallel on different buses.	High performance—critical devices use high-speed buses, while slower devices use lower-level buses.
Complexity	Simple and low cost.	Moderate complexity and cost.	Most complex and expensive design.
Scalability	Limited, as adding more devices increases contention and reduces speed.	Better scalability than single-bus systems.	Highly scalable and modular.
Example Systems	Early microcomputers and simple embedded systems.	Modern PCs with separate memory and I/O buses.	Advanced computer systems—e.g., PCs with PCIe (high-speed), USB (I/O), and local buses forming a hierarchy.
Typical Buses Used	System Bus (Address, Data, Control).	Memory Bus, I/O Bus.	Front Side Bus (CPU–Memory), PCI/PCIe Bus, USB/SATA Bus (I/O).

Q: Explain the working of Direct Memory Access (DMA) with a neat block diagram.

In modern computer systems, transferring data between input/output devices and memory can be a slow process if the CPU is required to manage every step. To address this, a Direct Memory Access (DMA) Controller is utilized.

A Direct Memory Access (DMA) Controller solves this by allowing I/O devices to transfer data directly to memory, reducing CPU involvement. This increases system efficiency and speeds up data transfers, freeing the CPU to focus on other tasks. DMA controller needs the same old circuits of an interface to communicate with the CPU and Input/Output devices.



All registers in the DMA appear to the CPU as I/O interface registers. Therefore, the CPU can both read and write into the DMA registers under program control via the data bus.

The figure below shows the block diagram of the DMA controller. The unit communicates with the CPU through the data bus and control lines. Through the use of the address bus and allowing the DMA and RS register to select inputs, the register within the DMA is chosen by the CPU. RD and WR are two-way inputs. When BG (bus grant) input is 0, the CPU can communicate with DMA registers. When BG (bus grant) input is 1, the CPU has relinquished the buses and DMA can communicate directly with the memory.

The CPU initializes the DMA by sending the given information through the data bus.

- The starting address of the memory block where the data is available (to read) or where data are to be stored (to write).
- It also sends word count which is the number of words in the memory block to be read or written.
- Control to define the mode of transfer such as read or write.
- A control to begin the DMA transfer

Modes of Data Transfer in DMA

There are 3 modes of data transfer in DMA that are described below.

Burst Mode

- In Burst Mode, the DMA controller takes full control of the system bus and transfers the entire block of data in one go.
- The bus is not handed back to the CPU until the entire data transfer is complete.
- This mode is efficient for large data transfers but can delay CPU operations.

Transparent Mode

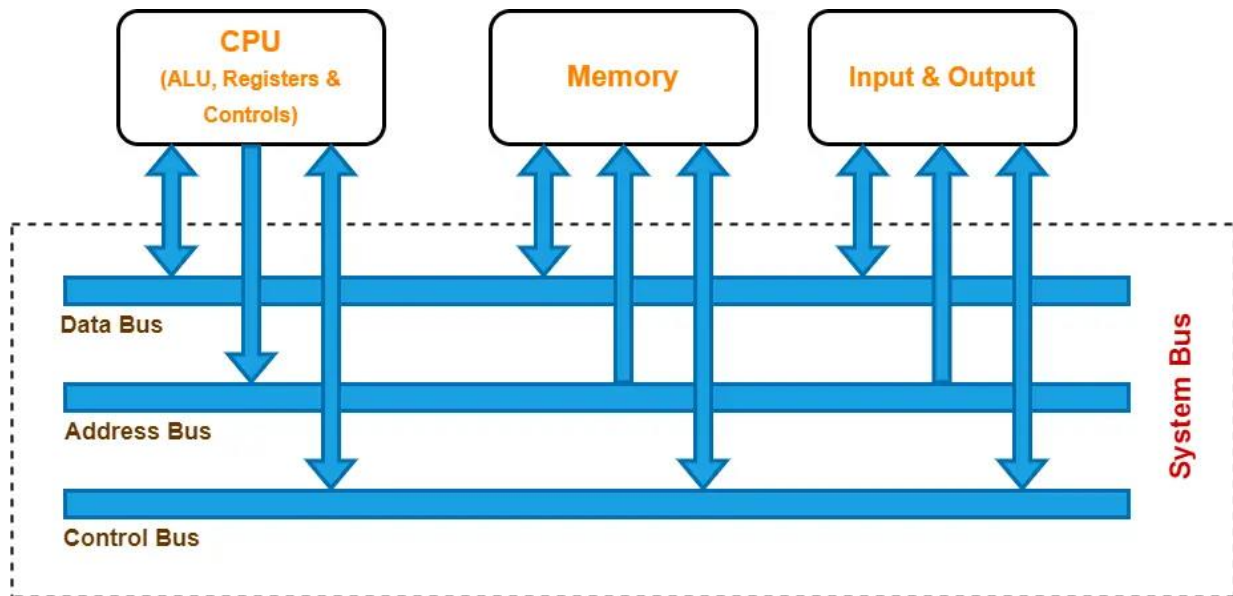
- In Transparent Mode, the DMA controller transfers data only when the CPU is not using the system bus.
- It effectively sneaks in transfers during idle CPU cycles, ensuring the CPU is never interrupted.
- Best when CPU performance is critical and some delay in data transfer is acceptable.

Cycle Stealing Mode

- In Cycle Stealing Mode, the DMA controller transfers one byte (or word) at a time and then releases control of the bus back to the CPU.

- This mode generates frequent bus requests but allows the CPU to execute instructions in between DMA transfers.
- Useful when the DMA task is important but should not entirely block the CPU, such as in audio or video streaming.

Q: Explain the system bus architecture with neat diagrams. Describe data bus, control bus, and address bus.



1. Data Bus

- The **data bus** carries **actual data or instructions** between the CPU, memory, and I/O devices.
- It is **bidirectional**, meaning data can flow in both directions.
- The **width** (number of lines) of the data bus determines how much data can be transferred at one time.
 - Example:
 - An 8-bit data bus can transfer 1 byte at a time.
 - A 32-bit data bus can transfer 4 bytes at a time.
- **Function:** Transfers operands, results, and instructions.

2. Address Bus

- The **address bus** carries the **addresses of memory locations or I/O ports** where data is to be read from or written to.
- It is **unidirectional** — data flows **only from CPU to memory or I/O**.
- The **width** of the address bus determines the **maximum addressing capacity** of the system.
 - Example:
 - A 16-bit address bus can address $2^{16} = 65,536$ (64 KB) locations.
 - A 32-bit address bus can address $2^{32} = 4,294,967,296$ (4 GB) locations.

- **Function:** Specifies the source or destination of data.

3. Control Bus

- The **control bus** carries **control and timing signals** that manage and coordinate the activities of the system.
- It is generally **bidirectional** since control signals can go both ways.
- These signals ensure correct timing and sequence of operations.
- **Common control signals include:**
 - **Memory Read (MEMR)** – CPU reads data from memory
 - **Memory Write (MEMW)** – CPU writes data to memory
 - **I/O Read, I/O Write**
 - **Clock signal**
 - **Interrupt Request (IRQ)**
 - **Bus Request/Grant, Reset**

Q: Compare PCI, USB, and SCSI interfaces in terms of speed, cost, and use cases.

Feature	PCI (Peripheral Component Interconnect)	USB (Universal Serial Bus)	SCSI (Small Computer System Interface)
Full Form	Peripheral Component Interconnect	Universal Serial Bus	Small Computer System Interface
Connection Type	Parallel bus inside the computer (internal)	Serial bus for external peripherals	Parallel interface (some modern versions are serial – SAS)
Data Transfer Speed	133 MB/s (PCI), up to 32 GB/s for PCIe (modern)	USB 2.0: 480 Mb/s USB 3.0: 5 Gb/s USB 3.2/4: up to 40 Gb/s	SCSI: 5–80 MB/s Ultra320 SCSI: 320 MB/s Serial Attached SCSI (SAS): up to 12 Gb/s
Cost	Moderate (depends on version)	Low-cost and widely available	Expensive , enterprise-grade
Cable Length / Distance	Short (inside motherboard, a few cm)	Up to 5 m (USB 2.0), 3 m (USB 3.0), longer with hubs	Up to 12 m (depends on type)
Typical Devices Connected	Internal devices like graphics cards, network cards, sound cards, SSDs	External devices such as mouse, keyboard, flash drive, printer, camera	High-performance devices like servers, hard disks, RAID arrays, scanners
Hot Plugging	Not supported in classic PCI (supported in PCI Express hot-plug systems)	Supported (plug and play)	Not supported in older versions; limited support in newer ones
Ease of Use	Requires installation inside the system	Very user-friendly , plug-and-play	Requires configuration and setup (device IDs, termination)

Feature	PCI (Peripheral Component Interconnect)	USB (Universal Serial Bus)	SCSI (Small Computer System Interface)
Bus Width	32-bit or 64-bit	Serial (1-bit)	8-bit or 16-bit (parallel), Serial in SAS
Use Case Summary	High-speed internal communication between CPU and expansion cards	Flexible, low-cost external peripheral connection	High-performance storage and server interfaces
Example Devices	Graphics card, NIC, sound card	Mouse, keyboard, flash drive	Hard drives, CD/DVD drives, RAID systems